DS

University of Applied Sciences Hamburg	**Bus Systems**	Hochschule für Angewandte Wissenschaften Hamburg *Hamburg University of Applied Sciences*
Group No: 1	**Lab 1** Room 8.65	Efe Kardes Matriculation No: 2757072
Date: 09/05/2026		Emir Avni Sahin Matriculation No: 2756541
Professor: Ratmus Rettig		Stanislaw Chilecki Matriculation No: 2761057
Temperature sensor with 1-wire interface (DS18B20)		

Report History

		Date		Remarks
Report 1	received			
	checked			
	result	o.k.		
		n.g.		

Prof. .....

Contents

1 Introduction

The main goal of this laboratory experiment was to get some experience with how digital sensors actually talk to a computer and how they react to real-world temperature changes. Instead of just reading a number off a screen, we wanted to observe the digital communication and the physical process of the heating and cooling cycles.

The three main objectives were:

- **Digital Communication:** We utilized a DS18B20 digital sensor interfaced with an Arduino Nano. Our first task was to use an oscilloscope to capture the actual electrical pulses, so called the telegrams, sent between the Arduino (Master) and the sensor (Slave). By analyzing these waveforms, we aimed to decode specific 1-Wire commands, such as the "Match ROM" sequence, and identify the sensor's unique family code.
- **Data Acquisition:** To handle the results, we had to establish a complete measurement chain. This involved configuring the Arduino to sample data and interpreting a prepacked Python script to act as a data logger. This allowed us to grab temperature values via the USB serial port and save them into a CSV file for further analysis.
- **Thermal Dynamics:** Finally, we wanted to measure how fast the sensor actually responds to heat. By using a Peltier element as a thermal stimulus, we recorded the temperature as it rose and fell. The objective was to calculate the system's **time constant (τ)**, which describes the thermal inertia of our specific setup and tells us how long the system takes to reach a steady state.

By the end of this lab, we aimed to demonstrate a successful integration of hardware and software to not only read digital data but also to mathematically model the sensor's behavior in a professional environment.

2 Task 1: Hardware Setup and Wiring

2.1 Pins and Connections

To get the lab running, we connected the DS18B20 sensor to the Arduino Nano using a three-wire setup. We chose to use an external power supply instead of **parasite power** because it makes the sensor more stable when the Peltier element is working in full capacity.

The specific pins we used are listed in the table below:

Table 1: Hardware Connections Used in the Lab

Device & Wire	**Arduino Pin / PC Port**	**Used for**
Sensor - Red Wire	5V	Provides power
Sensor - Black Wire	GND	Ground connection
Sensor - Yellow Wire	Digital D10	Data line (1-Wire Bus)
5kΩ Resistor	Junction (DQ to 5V)	Pull-up for the data line
Arduino USB Cable	PC USB Port	Sends data to Python
Python Script	COM5	Logs the data to CSV

2.2 Sensitivity to Touch

To test if our setup was working correctly before starting the Peltier experiment, we touched the DS18B20 sensor with our fingers. We observed the following changes in our system:

- **Sensor Level:** The internal ADC inside the DS18B20 detected the increase in thermal energy. This updated the temperature values stored in the first two bytes of the Scratchpad memory.
- **Data Stream:** Looking at the serial monitor in the Arduino IDE (and our Python console), we saw the floating-point values jump from room temperature (≈24°C) toward body temperature (≈34°C) within a few seconds.
- **Python Logging:** Our Python script `01_Read_USB_CSV_TimeStamped.py` immediately captured this rise. In the CSV file, we could see the timestamps showing a steady upward trend, proving that our measurement chain had low latency and was sensitive enough to detect small heat changes.

2.3 Data Acquisition Process

The measurement starts inside the DS18B20, which converts the physical heat into a digital number. The Arduino acts as the Master , asking the sensor for this value. Once the Arduino gets the raw bits, it does some math to turn them into a Celsius temperature we can read.

To get this data to the laptop, the Arduino sends it as a simple text string through the USB cable. We used a Python script on the laptop to listen to this connection. The script grabs each temperature reading and immediately adds a "Timestamp" using the laptop's internal clock. This is important because the Arduino doesn't know what time it is; the Python script handles the timing so we know exactly when each measurement happened.

2.4 Touch Test

To check if the temperature sensor reacts correctly, the sensor was touched by hand during measurement. The temperature increased from 25.06 °C to 26.94 °C within about 14 s. This confirms that the DS18B20 was working and that the measured values were sent correctly to the laptop.

Table 2: Temperature increase while touching the sensor

Measurement	Time [s]	Temperature [°C]
0	0.00	25.06
1	2.48	25.19
2	3.77	25.31
3	5.05	25.44
4	6.33	25.63
5	7.62	25.81
6	8.90	26.00
7	10.18	26.25
8	11.47	26.50
9	12.75	26.69
10	14.03	26.94

This simple "touch test" confirmed that our wiring was solid, the pull-up resistor was working, and our software was correctly logging real-time data.

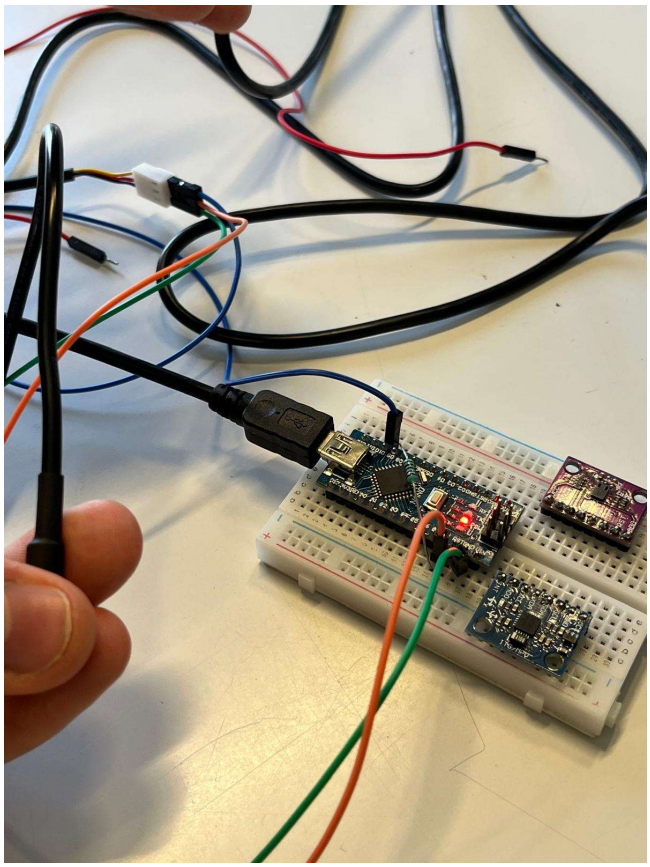


Figure 1: Touch Test

2.5 Hardware Block Diagram

While the DS18B20 supports parasite power via the internal diodes and C_{PP}, this lab utilized a dedicated **USB 5V supply** to the V_{DD} pin. This configuration was chosen to ensure voltage stability during the Peltier thermal experiments.

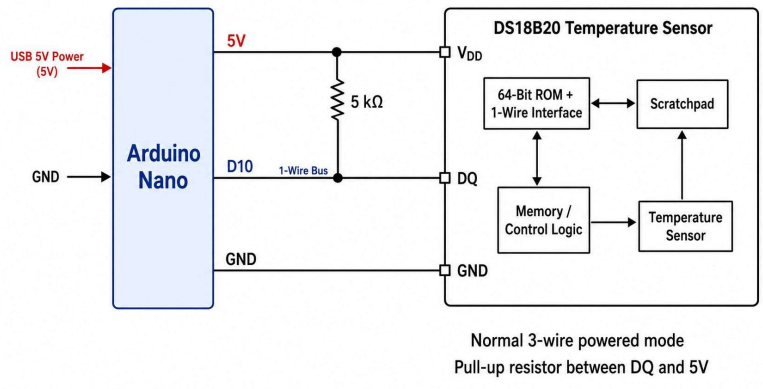


Figure 2: Hardware block diagram of the DS18B20 measurement system.

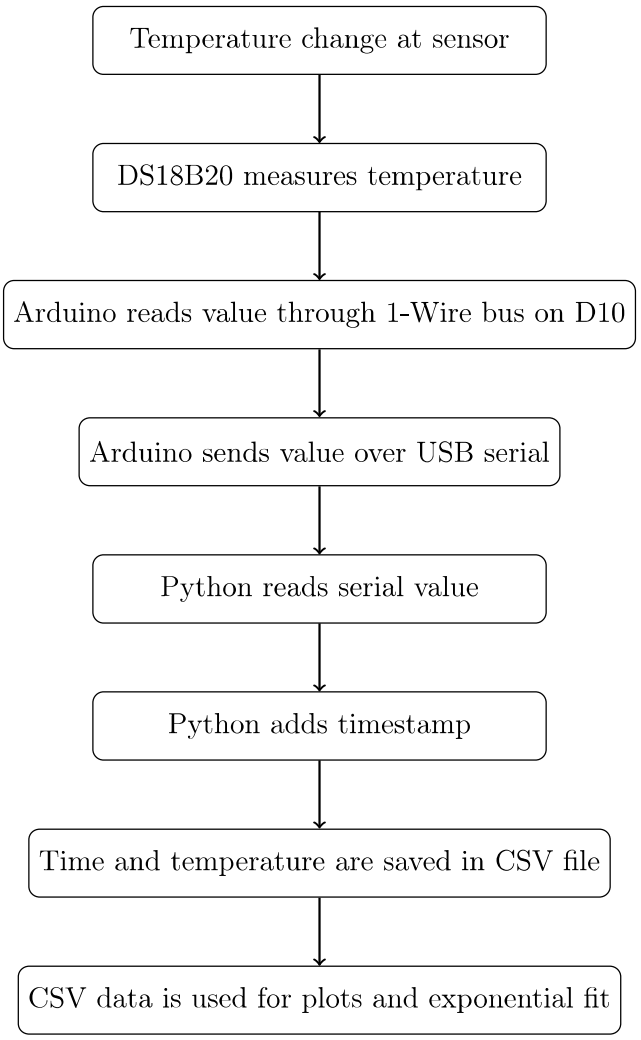


Figure 3: Data flow from temperature measurement to CSV file and plots.

3 Task 2: Investigation of the 1-Wire Interface

3.1 Explanation

The DQ line of the DS18B20 was measured with an oscilloscope. The probe was connected to DQ, and the ground clip was connected to GND.

The DQ line is normally HIGH because of the 5 kΩ pull-up resistor. During communication, the Arduino or the sensor pulls the line LOW. These LOW pulses form the 1-Wire telegram.

A telegram starts with a reset pulse from the Arduino. The sensor answers with a presence pulse. After that, command and data bytes are sent. The bits are transmitted LSB first.

3.2 Setup

One of the most important parts of the wiring was the 5kΩ resistor. Because the sensor uses an open-drain data line, it can only pull the signal down to ground; it can't push it up to 5V on its own. The resistor sits between the data line (DQ) and the 5V line to **pull** the signal high so the Arduino can actually see the digital pulses.

We plugged the Arduino into the laptop via USB, which handled two duties: it gave the whole circuit 5V of power and let our Python script talk to the Arduino over the COM5 port to save our measurements.

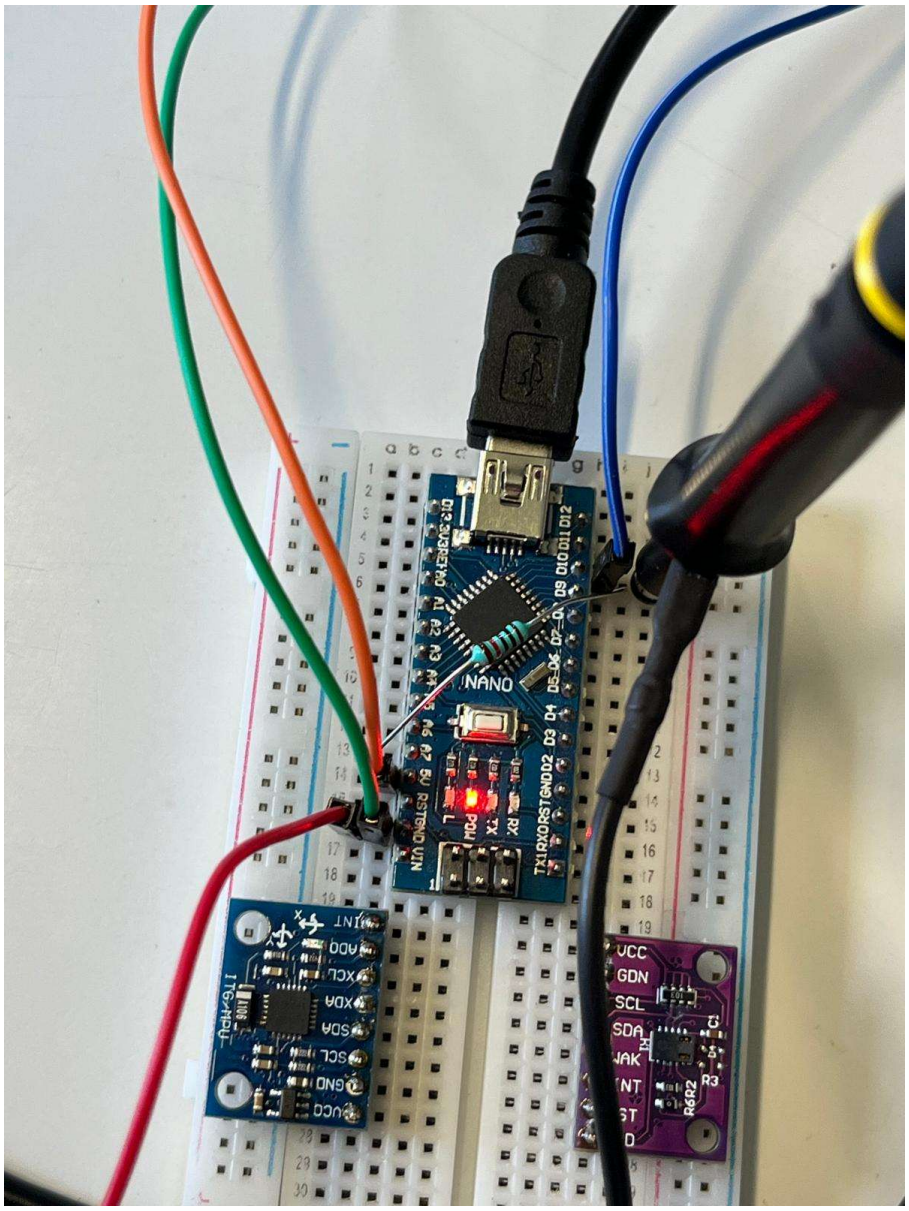


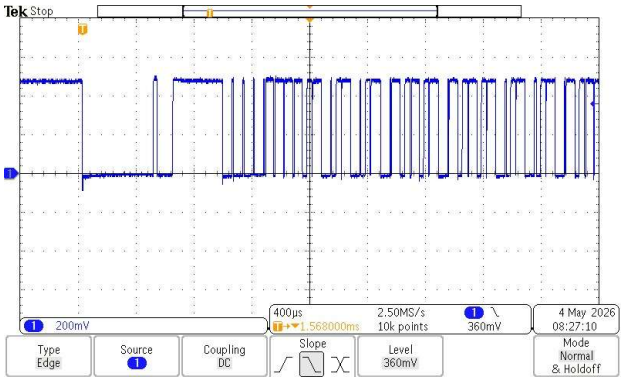
Figure 4: Setup on the Arduino Nano

Table 3: 1-Wire Telegram Parts

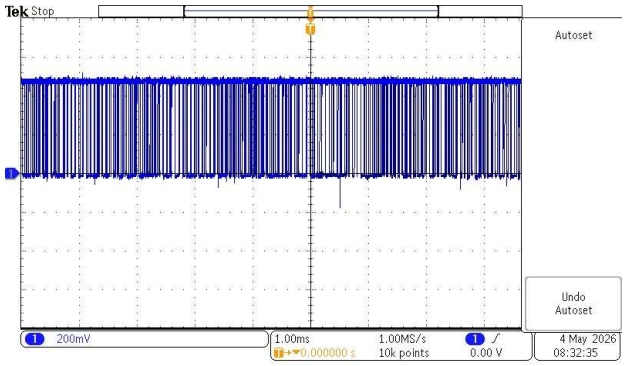
Part	Value / Type	Meaning
Reset pulse	–	Start of communication
Presence pulse	–	Sensor response
Command byte	From oscilloscope	Sensor command
Family code	`0x28`	DS18B20 family code
Bit order	LSB first	Lowest bit sent first

3.3 Decoding the Results

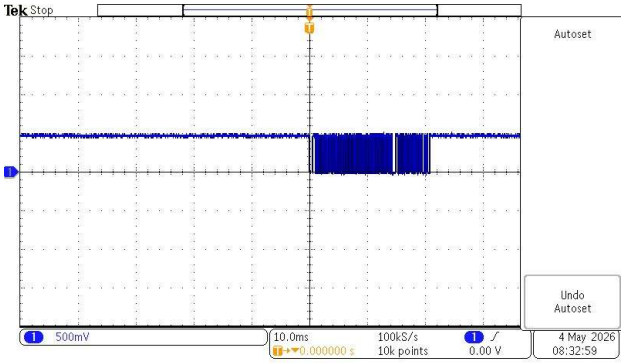
To fulfill the task requirements, we captured the 1-Wire bus activity at five different timescales ranging from 400μs/div to 400ms/div. These screen dumps allow for a detailed analysis of the communication frequency and command structure.



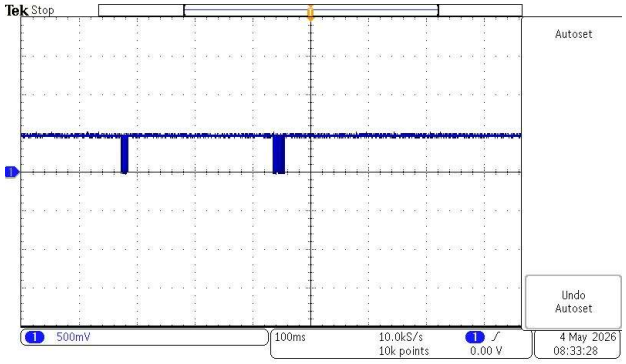
(a) 400μs/div



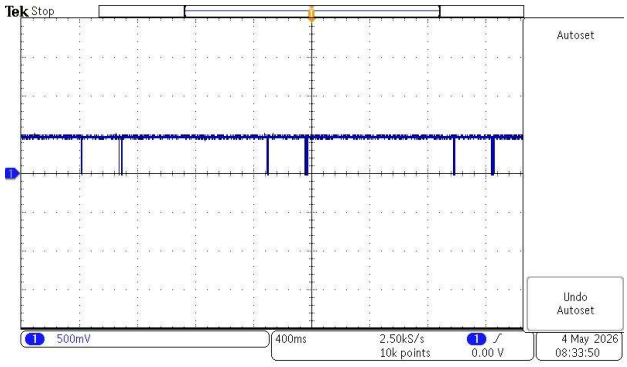
(b) 1ms/div



(c) 10ms/div



(d) 100ms/div



(e) 400ms/div

Figure 5: 1-Wire Telegrams captured at various time scales from 400μs to 400ms.

3.4 Protocol Analysis and Data Telegram Decoding

By analyzing the zoomed-in capture (Figure (a) at 400μs/div), we decoded the initial command sequence. Following the standard 1-Wire handshake (Reset and Presence), the Master initiated the following sequence:

1. **Command Phase:** The first byte observed was **0xF0 (Search ROM)**, followed by **0x55 (Match ROM)**. This confirms the Arduino successfully identified and addressed the specific 64-bit ROM ID of our sensor.

2. **Temperature Data Transfer:** When the Arduino sends the `Read Scratchpad [0xBE]` command, the sensor responds by transferring 9 bytes of data. The most critical "values transferred" are the first two bytes:

    - **Byte 0 (LSB):** Contains the fractional parts and lower integer bits.

- **Byte 1 (MSB):** Contains the higher integer bits and the sign bits.

3. **Bit Logic:** As shown in the 400μs/div capture, a logic '0' is represented by a long LOW pulse ($\approx$60μs), while a logic '1' is a very short LOW pulse followed by a return to HIGH. Because the protocol is **LSB-first**, we decoded the sequence `10101010` to identify the `0x55` command.

The interval between these telegrams, as seen in the 400ms/div capture, confirms the sampling rate configured in our Arduino code.

3.5 Data Transfer Frequency

We determined how often a new temperature value is sent over the bus by analyzing the timestamps in the saved measurement file. By looking at the time difference between consecutive rows in the data log, we found that a new temperature reading is transferred approximately every **1.3 second**.

This 1 Hz transfer rate is determined by the sampling loop in the Arduino code and the internal conversion time of the DS18B20 sensor. At its maximum 12-bit resolution, the sensor requires about 750ms to process a new temperature value before it can be read. The full list of timestamps and corresponding temperature values can be found in the **Appendix**.

4 Task 3: Dynamic Thermal Behavior

4.1 Explanation

The Peltier element was switched on and the temperature was recorded. Since the measurement started when heating started, the start time was set to

$$t_0 = 0.$$

A total of 1000 temperature values were recorded for the heating measurement.
The heating curve was fitted with

$$T(t) = T_0 + \Delta T \left(1 - e^{-t/\tau}\right).$$

T_0 is the initial temperature, ΔT is the temperature increase, and τ is the time constant.

4.2 Heating Curve

The temperature rises quickly at first and then becomes flatter near the final temperature.

4.3 Fit Results

The fitted function was

$$T(t) = 30.6247 + 12.1710 \left(1 - e^{-t/283.8125}\right).$$

Table 4: Heating Fit Parameters

Parameter	Value
T_0	30.6247 °C
ΔT	12.1710 °C
T_∞	42.7956 °C
τ	283.8125 s

The final temperature is

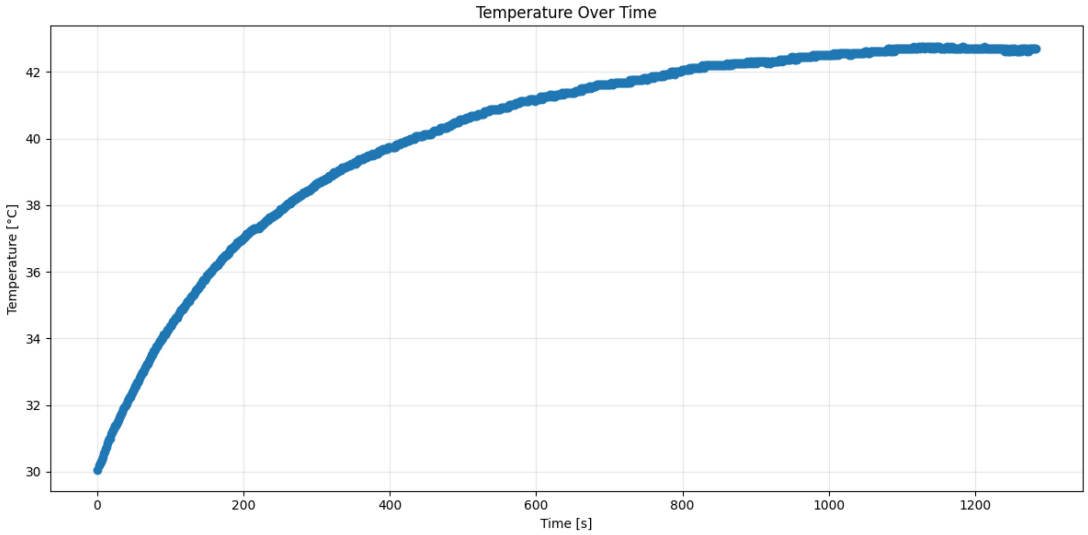


Figure 6: Measured heating curve.

$$T_\infty = T_0 + \Delta T = 42.7956\,^\circ\mathrm{C}.$$

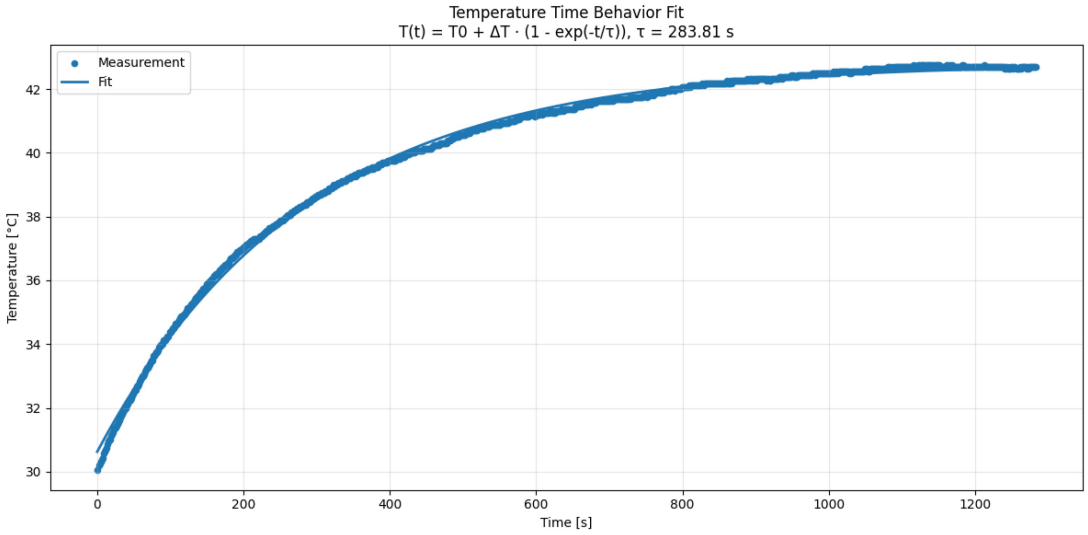


Figure 7: Heating curve with exponential fit.

4.4 Time Constant

The fitted time constant was

$$\tau = 283.8125\,\mathrm{s}.$$

The fitted function was

$$T(t) = 30.6247 + 12.1710\left(1 - e^{-t/283.8125}\right).-$$

For one time constant, $t = \tau$:

$$T(\tau) = 30.6247 + 12.1710\left(1 - e^{-1}\right)$$

$$T(\tau) = 30.6247 + 12.1710 \cdot 0.632$$

$$T(\tau) \approx 38.32\,^{\circ}\mathrm{C}.$$

So after about 284 s, the temperature is about 38.32 °C.

4.5 Fit Quality

Table 5: Heating Fit Quality

Metric	Value
Maximum deviation	0.5647 °C
Mean absolute deviation	0.1100 °C
Root Mean Square Error(RMSE)	0.1348 °C

The fit matches the measured curve well. The maximum deviation is below 0.6 °C.

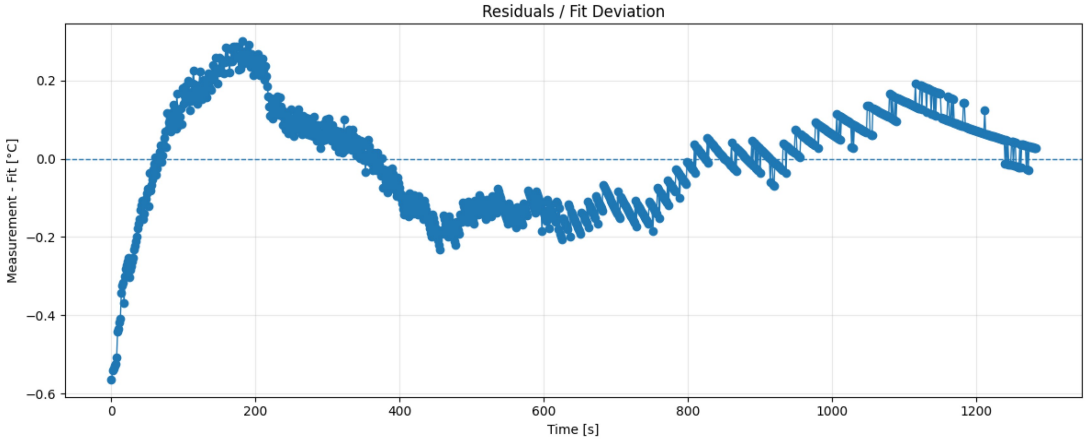


Figure 8: Difference between measurement and fit.

4.6 Possible Errors

Possible error sources are heat loss, weak thermal contact, sensor delay, room temperature changes, and limited sensor resolution.

5 Additional Cooling Evaluation

5.1 Explanation

The cooling curve was fitted with

$$T(t) = T_{\infty} + \Delta T e^{-t/\tau}.$$

The fitted function was

$$T(t) = 26.0093 + 16.3350 e^{-t/251.0614}.$$

The cooling time constant is smaller than the heating time constant. Therefore, the setup cooled slightly faster than it heated.

Table 6: Cooling Fit Results

Parameter	Value
T_∞	26.0093 °C
ΔT	16.3350 °C
Initial temperature	42.3443 °C
τ	251.0614 s
Maximum deviation	0.8941 °C

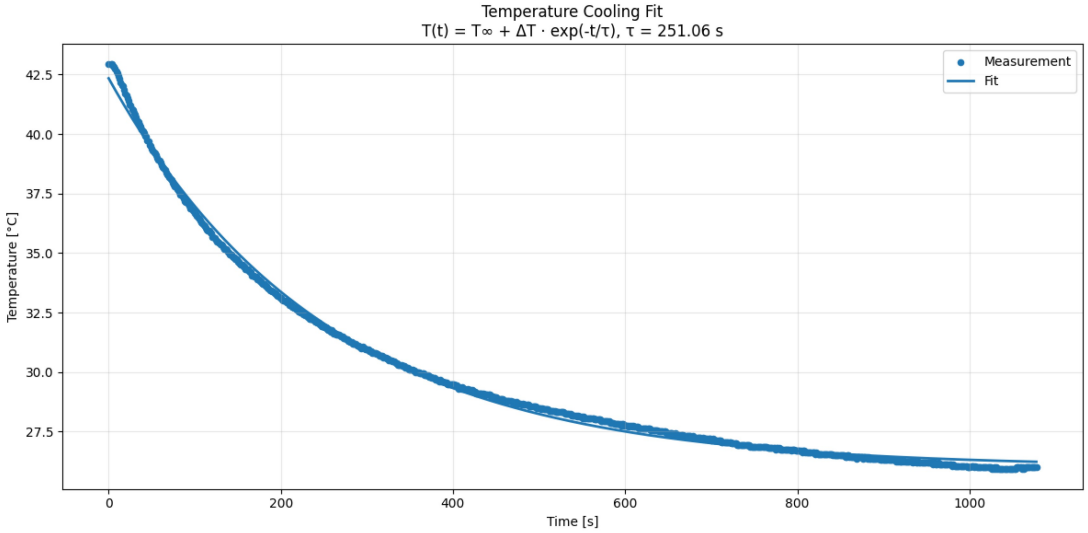


Figure 9: Cooling curve with exponential fit.

6 Summary

The DS18B20 sensor was measured successfully. The 1-Wire signal was observed on the DQ line. For heating, the fitted model was

$$T(t) = T_0 + \Delta T \left(1 - e^{-t/\tau}\right).$$

The fitted values were

$$T_0 = 30.6247\,^\circ\text{C}, \quad \Delta T = 12.1710\,^\circ\text{C}, \quad T_\infty = 42.7956\,^\circ\text{C}, \quad \tau = 283.8125\,\text{s}.$$

The maximum deviation was

$$0.5647\,^\circ\text{C}.$$

The exponential model fits the heating measurement well.

A Appendix: Transmission Interval

The first timestamps were used to estimate how often a new temperature value was transmitted. The delay between two values was calculated from the timestamp difference:

$$\Delta t = 3.77 - 2.48 = 1.29\,\text{s}$$

$$\Delta t = 5.05 - 3.77 = 1.28\,\text{s}$$

Table 7: First measured timestamps

Measurement	Time [s]	Temperature [°C]
0	0.00	30.06
1	2.48	30.19
2	3.77	30.25
3	5.05	30.31
4	6.33	30.37

Therefore, a new temperature value was transmitted about every

$$\Delta t \approx 1.3\,\mathrm{s}.$$